HARVARD
GRADUATE SCHOOL OF EDUCATION

COURSE SYLLABUS

S-062: Computer-Assisted Text Analysis for Social Research

Fall 2026

Time	Tue/Thu, 10:30-11:45 AM	**Instructor**	Zachary Lim
Place	Larsen G01	**Office**	Larsen 420
Website	Canvas site	**Email**	zlim@g.harvard.edu
Prerequisites	None	**Phone**	[REDACTED]
		Office Hours	Thu, 12:00-1:00 PM; Mon/Fri, 2:00-3:00 PM; or by appointment

Contents

1 Overview **2**
- 1.1 Prerequisites . . . 3
- 1.2 Objectives . . . 3
- 1.3 Structure . . . 4

2 About the Instructor **6**
- 2.1 Teaching Philosophy . . . 6

3 Assessments and grading **6**
- 3.1 Ungrading: learning goals, evidence, and the final conference . . . 6
- 3.2 Course expectations . . . 7
  - 3.2.1 Pre-session preparation . . . 8
  - 3.2.2 Attendance and participation . . . 8
  - 3.2.3 Working with your own corpus . . . 8
  - 3.2.4 Unit assignments and the final project . . . 9
  - 3.2.5 Instructor check-ins . . . 9
  - 3.2.6 Collaboration, LLM use, and academic integrity . . . 9

4 Student support **10**
- 4.1 Accessibility . . . 10
- 4.2 Getting Help . . . 10

5 Course calendar **11**

6 Detailed unit breakdown **12**
- Unit 1: Course introduction & framework . . . 12
- Unit 2: Data organization & corpus structure . . . 13
- Unit 3: Cleaning & sparse translations . . . 14
- Unit 4: What an $n \times m$ matrix enables . . . 16

Unit 5: Researcher-defined measurement with sparse representations 17
Unit 6: Unsupervised learning for dimensionality reduction: SVD/LSA, topic models, UMAP, & clustering 18
Unit 7: Predictive supervised learning with sparse representations 19
Unit 8: Downstream analysis with imperfect measurements 20
Unit 9: Learned dense representations: distributional semantics, Word2Vec, & pretrained encoders 21
Unit 10: Linguistic feature engineering: NER, dependency parsing, coreference, SRL, & WSD . 23
Unit 11: Computer-assisted qualitative inquiry and geometric coding 24
Unit 12: Course synthesis & large language models 25

1 Overview

Computer-assisted text analysis (CATA) is the practice of *translating* text into documented artifacts that support systematic analysis. This course introduces applied methods for creating, evaluating, and using those translations in social research.

For social scientists, **texts are an invaluable source of data**. Whether in the form of primary sources, interviews, assessments, or other records of human communication, language is one of the most natural and important ways people preserve information, express meaning, and navigate social life. Thus, being able to study texts systematically is a powerful addition to a social scientist's research toolkit.

Computer-assisted text analysis (CATA) is the practice of using computational tools to study texts at varying scales without giving up the responsibilities of reading, interpretation, and judgment. This course introduces CATA as an applied methodological framework for social scientists who want to work systematically with language data. Students will learn how to organize textual data, discover patterns across many documents, identify similar or unusual cases, build systematic measures from text, and compare those measures across groups, settings, or time. We will learn to accomplish these tasks in ways that can scale to datasets with millions of texts, and learn how to make defensible statements about our results.

The course is especially well suited for students who already have text data, research materials, or emerging text-based projects they want to work through, though an existing project is not required. **Cross-registered students are warmly welcome**, and auditors are welcome where permitted by program and university policies. Prior experience with Python, data science, or computational methods is helpful but not required; curiosity, care, and willingness to work through unfamiliar tools matter more.

The course is organized around a guiding metaphor: computational methods **translate** text into different forms. There is generally no uniquely correct computational representation of a substantive text, but translations can be more or less useful for a given research question. For example, they can make patterns easier to see, help us compare texts, perform measurements, make predictions, and navigate large collections. Throughout the course, we will evaluate each translation using the **AVAIL** mnemonic:

- **Affordances:** What does this translation enable or make easier?
- **Validation:** How do we know the translator got right what it set out to represent?
- **Additions:** What information enters from outside the translator's immediate input artifact or artifacts?
- **Impositions:** What structure does the translator assume or impose on the data?
- **Losses:** What information available in the input is lost, collapsed, or obscured?

AVAIL provides a recurring framework for thinking critically about what each translation does and whether

it is appropriate for a substantive research purpose.

Because this is an applied methods course, we will not spend much time on mathematical derivations or technical internals (though the math is beautiful, and I am always happy to discuss it further during office hours). Instead, we will focus on understanding what different methods allow us to do, the choices involved in using them, and how to interpret and evaluate their results. To make this possible, we will use a software package I created, **Text Analysis Lab (TeAL)**, which provides methodological infrastructure for systematic and scalable computer-assisted text analysis. TeAL uses provenance-aware artifacts and sensible defaults to reduce technical boilerplate while keeping source material, translations, metadata, and context traceable.

In short, **my goal is for students to leave this course feeling more confident and capable of using text as a source of data**. This includes not only being able to analyze existing textual materials, but also being able to collect and work systematically with more open-ended data, giving the people and phenomena we study more room to surprise us rather than requiring every relevant category or distinction to be specified in advance.

1.1 Prerequisites

There are **no formal prerequisites** for this course. Students are not expected to arrive with prior experience in Python or data science. That said, the course does assume a willingness to learn new technical tools, to work carefully through unfamiliar material, and to put in steady effort across the semester.

Because the course includes hands-on computational work, students should have regular access to a reasonably capable laptop. It does not need to be especially powerful, but it should be able to run Python and handle moderately sized datasets without difficulty. Students planning to work with larger personal datasets may need somewhat more memory or computing capacity.

Prior coursework in data analysis, statistics, or computer science is helpful but not required. Students without a coding background are welcome; I will provide most of the code you need (especially if you are not using your own dataset), and some class time will be devoted to working on writing code. However, **if you've never written code before, you should also be prepared to put in additional time and effort to become comfortable with that**. I (and the TF, if we get one) will be available for office hours and we will go over some strategies for how to learn how to code independently.

Finally, **students from outside the Harvard Graduate School of Education (HGSE) are very welcome**! I believe the quality of learning will only be improved with increased intellectual diversity.

If you have any questions about whether S-062 is right for you, **please reach out**.

1.2 Objectives

By the end of the course, students should be able to:

1. **organize and document** collections of text so that analyses remain traceable to their original sources;
2. **explain and evaluate** computer-assisted text analysis as purposeful translation using AVAIL—Affordances, Validation, Additions, Impositions, and Losses;
3. **construct and compare** multiple computational representations of the same texts and justify which are appropriate for a substantive research task;
4. **use** those representations for comparison, search, visualization, exploration, strategic qualitative reading, and model-assisted analysis;
5. **build and evaluate** both researcher-defined and predictive measures from text while distinguishing

constructs, measures, and measurements;

6. **analyze** imperfect text-derived measurements across documents, groups, settings, or time while accounting for systematic error, bias, and uncertainty;
7. **validate** computational translations directly or downstream by returning to source context and, where appropriate, comparing outputs with higher-quality measurements or judgments;
8. **complete** a reproducible text-analysis project that composes appropriate translators into a defensible analytic pipeline using course data and, where appropriate, a corpus connected to students' own research.

1.3 Structure

This course follows a **flipped classroom** format. Before each Tuesday theory session, students are expected to watch the short lecture videos, review any assigned readings or other preparation materials, and complete the pre-session preparation form. The form is graded for completion rather than accuracy and gives students an opportunity to identify questions or points of confusion. I will use the responses to begin Tuesday's session by addressing common misconceptions and recurring questions before we move into discussion and application.

Most units of the course have four components:

- short asynchronous videos and other conceptual preparation, completed before Tuesday;
- a short pre-session preparation form, also completed before Tuesday;
- a Tuesday theory session that clarifies and deepens the week's main concepts; and
- a Thursday application session focused on implementation, coding, and hands-on work with a guided notebook.

The videos are intentionally conceptual rather than software walkthroughs. Step-by-step TeAL implementation will usually live in short guided notebooks that students can reuse during Thursday practica and later assignments. Most weeks, we will aim to get through one unit. Figure 1 shows the chronological progression of all twelve units. Its colors classify units into three recurring conceptual categories—**Foundational concepts**, **Ways of translating**, and **Using translations to do research**—rather than dividing the course into three sequential phases. Refer to the course calendar in Section 5 for full dates.

The course will use a shared dataset for examples, demonstrations, and common practice. Students will also apply the week's workflows in their own work as the semester progresses. More detail on expectations appears in the assessment section.

We will also use **Text Analysis Lab (TeAL)** throughout the course. TeAL is a Python package I created as methodological infrastructure for systematic and scalable computer-assisted text analysis. It brings together the main tools and workflows used in the course while tracking source artifacts, translators, translations, metadata, context, and provenance. You can think of TeAL as an operationalization of the CATA framework: it handles much of the bookkeeping and technical boilerplate with sensible defaults so that students can focus on methodological choices, validation, and interpretation.

At the end of most units, students will submit a brief memo-based assignment. These weekly memos are cumulative: each one builds on earlier work and contributes to the final project completed at the end of the course.

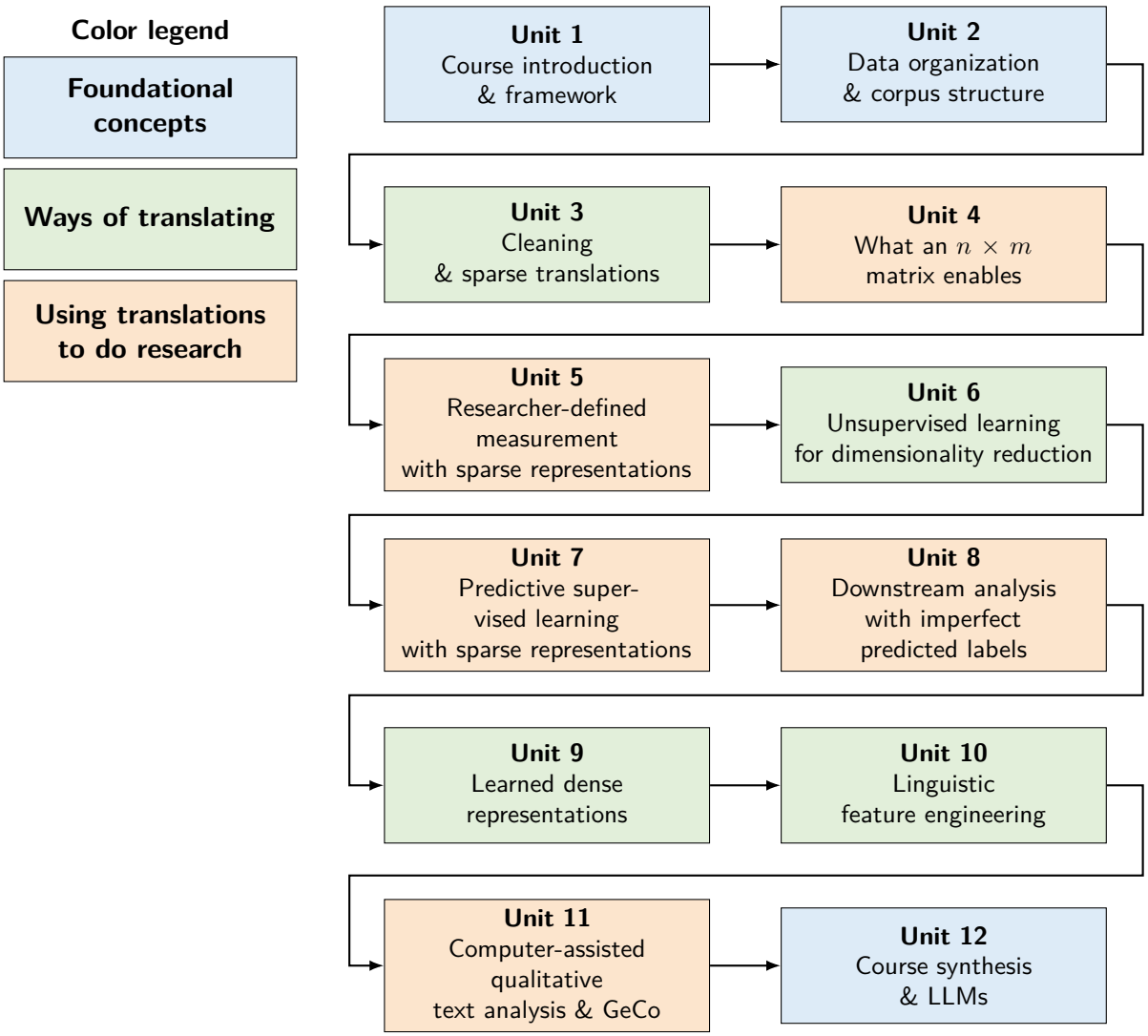


Figure 1: Chronological course progression and conceptual classification of units

2 About the Instructor

Teaching this course has been a long-time dream of mine! Computer-assisted text analysis (CATA) is my main methodological specialization, and much of my research involves building tools, frameworks, and workflows to make text analysis more rigorous, transparent, and useful for social scientists. I believe CATA has a great deal of untapped potential for social science research, and I am excited to explore how these tools can help advance your own work.

I am an interdisciplinary scholar by temperament and training, and I genuinely enjoy classrooms where different perspectives, methods, and fields are in conversation (you will see this reflected in our course readings). Outside of academic work, I also practice Tai Chi in the community, and I hope we may have a chance to share that as well.

2.1 Teaching Philosophy

More generally, I approach teaching as a fundamentally *collaborative* process. My role as the instructor is to bring *expertise*, *structure*, and *genuine enthusiasm* to the course, and to *connect* the material to your own goals as a student and researcher. Your role is to *engage* seriously with the material, *communicate* openly, and be *willing to work* through ideas that may not feel easy at first.

I also believe that learning is not only a relationship between instructor and student, but something shaped by the classroom community. *Your peers are a resource.* They bring different backgrounds, questions, and strengths, and part of the purpose of the flipped format is to create space for that collective learning. I am excited to learn from everyone, and I hope you are as well.

3 Assessments and grading

This course adopts an **ungrading** approach to assessment: rather than accumulating points across assignments, attendance, and participation, students will develop their own learning goals, work toward them throughout the semester, and make an evidence-based case for their final grade in an individual conference with me. The final project and the student's ability to explain and defend their work will be important evidence, but we will consider learning and engagement across the semester as well.

This does not mean that preparation, attendance, participation, assignments, and check-ins no longer matter. They remain **course expectations**, and the teaching team will continue to monitor them and provide feedback. They are opportunities to learn, to contribute to our shared work, and to build evidence for the case students will eventually make. The sections below explain first how the grading process works, then what is expected of students as the semester unfolds.

3.1 Ungrading: learning goals, evidence, and the final conference

Choose your learning goals. This course deliberately spans several areas—Python programming, natural language processing, data science and machine learning, and social science research methods—because students enter with different backgrounds, interests, and research needs. No one is expected to master everything. Instead, students will complete a **learning-goals worksheet** that introduces possible goals in each area, invites them to assess where they started, and asks them to select and prioritize the areas in which they want to grow. Students may propose additional goals that are relevant to the course. Choosing priorities also means deciding what not to prioritize; thoughtful focus is preferable to feeling overwhelmed by everything at once.

Students have substantial autonomy in selecting their goals and the standards by which they hope to assess their progress. I reserve the right to ask students to revise their goals if I feel that they are not sufficiently

ambitious or if they fall outside the scope of the course/what I can support, but I will only exercise this authority sparingly. Goals can also evolve as a student's understanding and research project develop.

Plan backward from the case you want to make. The worksheet will ask students to imagine the end of the semester: What would a convincing case for an A look like? What would a convincing case for a B look like? What would they need to know, do, or produce to substantiate either case? Students will then work backward to identify the activities, habits, and project milestones that could build that evidence. This is an exercise in **backward design** and **evidence-based reasoning**, not a binding contract or a checklist of points to earn. A student can take on an ambitious goal, encounter genuine obstacles, and still make an excellent case by demonstrating substantial learning and thoughtful judgment.

These goals are also meant to help the teaching team support students more effectively. We will use them to tailor guidance and feedback where possible. For example, students who prioritize programming competence should seek opportunities to write and troubleshoot more of their own code in the weekly assignments; the TF can then give feedback on that work. Students pursuing other goals can use the same assignments to develop and demonstrate those capabilities. The aim is to make course activities more useful for each student's chosen trajectory, not to create a different point system for each student.

Present and defend your learning. At the end of the course, each student will complete a final project and meet individually with me. In this conference, the student will present their project, answer questions about their methodological decisions and understanding, and revisit the learning-goals worksheet. They will then **propose a final course grade and make a case for it**, drawing on the project, their development, their work throughout the semester, and any other relevant evidence. I may ask questions, challenge a claim, or suggest a higher or lower grade based on that evidence. My role is to assess whether the case is convincing and the claimed learning has been demonstrated; I retain responsibility for assigning the final grade.

What counts as evidence? Before each conference, I will review the student's record across the semester, including attendance, participation, preparation, unit assignments, feedback, and check-ins, alongside the final project. The teaching team will continue to keep track of this work. None of these activities has a predetermined numerical weight, and no single attendance count or assignment tally automatically determines a letter grade. For instance, exceptional demonstrated competence and an excellent independent project may outweigh a limited attendance record. Conversely, if a student builds their case around sustained effort and engagement, that claim should be supported by what they actually did. Students are encouraged to identify and explain other relevant evidence of learning, including substantial work completed outside class. The question is not simply what grade a student wants, but what grade the evidence can reasonably support.

3.2 Course expectations

The following activities are the regular structure through which students can learn, practice, receive feedback, and build their final case. They are **expected**, even though they do not carry individual point values or automatic grade penalties. Students should take these expectations seriously and communicate with the teaching team when circumstances interfere. We will use missed work or attendance as information for supporting students and interpreting their eventual cases, not as entries in a weighted grading formula.

3.2.1 Pre-session preparation

Before each Tuesday theory session, students should complete a short online preparation form after watching the assigned videos and reviewing any other preparatory materials. The form will usually include roughly 5–10 brief questions, depending on the amount of material, along with a prompt for outstanding questions, points of confusion, or concepts that need clarification.

These forms are **not graded for accuracy or completion**. Their purpose is diagnostic: students should use them to check their own understanding without worrying about being penalized for an incorrect response. I will use the responses to identify common misconceptions, difficult concepts, and recurring questions, which will help structure the beginning of Tuesday's session before we move into discussion and application.

Completing the preparation form is also part of students' responsibility to one another. Everyone should arrive prepared enough to contribute productively to collaborative work rather than relying on classmates to carry the conceptual load. Unless otherwise announced, the form should be submitted before the corresponding Tuesday theory session.

3.2.2 Attendance and participation

Students are expected to attend and participate in class sessions. Because this course is built around a flipped and discussion-based format, much of the learning happens in the classroom through collective interpretation, coding practice, troubleshooting, and discussion. Attending regularly is a valuable way to practice, receive feedback, and contribute to others' learning, although it is not an automatic condition for earning any particular grade.

Participation includes asking questions, contributing to discussion, working seriously during in-class activities, and engaging constructively with peers. Students should come to class ready to discuss the week's material, including questions or points of confusion that arose while watching the videos or working through the code.

Attendance will continue to be tracked. If you expect to miss class or arrive late, please let me know in advance whenever possible. A student's pattern of attendance and participation can inform our final discussion, but there is no automatic deduction for a missed class.

Professional classroom behavior also matters. **There will be times when I ask students to put laptops away** (yes, the irony of this given the course title is not lost on me) because computer use, even if course-related, can be distracting to others when we are trying to have a shared discussion. Students should therefore bring their computers to class, but also be prepared to set them aside when asked. Slides from the lecture videos will be made available, and students are welcome to take notes in whatever format works best for them.

3.2.3 Working with your own corpus

A central goal of this course is to help students apply computer-assisted text analysis to their own research. For that reason, **PhD students are expected to bring or identify their own corpus early in the semester and to use that corpus in their unit assignments and final project**. The aim is for the course to generate work that is not merely an exercise, but a meaningful step toward a conference paper, article, dissertation chapter, or other publication-ready analysis.

Master's and college students are also welcome to do this, but it is not expected. They may complete the course assignments using the shared dataset if they prefer.

3.2.4 Unit assignments and the final project

Most units of the course conclude with a short written assignment. These unit assignments will typically take the form of brief memos tied to the week's methods, translations, or analyses. Their purpose is to help students practice the course workflow in manageable pieces and to build incrementally toward the final project.

In most cases, students will work first from the shared course dataset and then apply the same workflow to their own corpus. Over time, these assignments should accumulate into a coherent body of work that can be revised, extended, and integrated into the final project. Students should also use these assignments to pursue their own goals: for instance, by writing more code themselves, concentrating on conceptual interpretation, or interrogating methodological assumptions. The teaching team will provide feedback with those goals in mind.

Unit assignments are **ungraded but cumulatively important**. They should be submitted according to the course calendar so that students can receive feedback while it is still useful and keep up with later material. There is no point deduction or fixed late-work penalty. Students can submit or revisit work after its intended deadline, though delayed submissions may limit the timeliness of feedback and make later work harder to complete. If you are falling behind, please communicate with the teaching team so we can make a productive plan.

The final project brings together the cumulative work of the semester into a single, coherent analysis. Students will use the methods and workflows developed across the course to produce a reproducible computer-assisted text analysis project. In most cases, this will involve combining and refining earlier unit assignments rather than starting from scratch. The final project is due **December 10, 2026, at 11:59 PM ET** and will form a central part of the final presentation and grade conference described above.

3.2.5 Instructor check-ins

Each student is expected to complete **two short individual check-ins** with me during the semester: one early in the term (or prior to it) and one around the midpoint. These meetings are meant to help clarify goals, identify obstacles early, and make sure that each student's work is developing in a manageable and productive direction. Students should come to these meetings prepared to discuss their progress, questions, and plans for the next stage of the course.

3.2.6 Collaboration, LLM use, and academic integrity

Students are encouraged to collaborate in the course by discussing ideas, troubleshooting code, and helping one another work through difficult concepts. Your peers are a resource, and collaborative learning is part of the pedagogy of this course. However, **any work you submit must ultimately be your own**. If another student, the instructor, or the TF made a meaningful contribution to your work, you should acknowledge that contribution clearly.

LLMs may be used in this course, but they do not reduce your responsibility for what you submit. If you use an LLM to implement an idea that is already yours—for example, to turn a clearly specified plan or pseudocode into mostly standard code—no formal attribution is required. But if you use an LLM because you do not know how to solve the problem yourself, and it supplies the substantive solution, approach, or analysis, then that use should be acknowledged. I discuss how I recommend you do this in class. The distinction is not whether AI was used, but whether the core intellectual work came from you. In all cases, you are responsible for checking, understanding, and defending everything you submit.

Plagiarism and misrepresentation of authorship are serious violations of academic integrity. Submitting

work that is not your own, or presenting ideas, analysis, or writing without clear attribution, is unacceptable. The same principle applies to AI-assisted work: if the ideas are not yours, or if you cannot explain and defend what you submitted, then the work does not count as your own. If I have reason to believe that submitted work does not reflect your own understanding, I may ask you to explain it. An inability to do so may be treated as a violation of course and school policy.

I will discuss responsible AI use in more detail during the course, including what kinds of attribution are appropriate in different cases. The broad principle is that these tools may assist your work, but they do not replace your judgment, understanding, or accountability.

4 Student support

4.1 Accessibility

From the HGSE website (as of May 14, 2026):

> The Harvard School of Graduate Education is committed to creating an environment that values disability as a part of the diverse range of identities within our student body. The mission of Accessibility Services, within the Office of Student Affairs, is to provide students with equal educational opportunity and full participation in all programming at HGSE and within the wider Harvard Community. Our goal is to remove educational and co-curricular barriers to education so that students may fulfill their professional aspirations during their time at HGSE. We also provide educational opportunities for the community to promote disability as a part of the diversity continuum. Accessibility Services engages students in an interactive process to determine reasonable accommodations. If you have questions, please contact hgsesupport@gse.harvard.edu.
>
> All accommodations are individualized, flexible, and determined based on the nature of the disability and the academic environment.

Details on how to request accommodations can be found on the HGSE website, although you should note that the process may take 1-3 weeks and accommodations do not apply retroactively, so take this into account when reaching out.

4.2 Getting Help

Learning in this course is cumulative, so it is important to seek help early rather than waiting until confusion has compounded. If something is unclear—conceptually, technically, or practically—please reach out.

Office hours are the first and best place to get help. They are there for you to use, and I strongly encourage you to come, even if your questions are still in an early or inexact form. In many cases, it can also be helpful when other students are present, since hearing each other's questions often clarifies issues that many people are working through.

You will also have opportunities to get help during class, especially during the application-focused sessions when we will have dedicated work time. Those moments are built into the course on purpose, so please make use of them.

If the listed office hours do not work for you, send me an email and we can try to arrange another time.

5 Course calendar

Most Tuesdays are theory sessions, and most Thursdays are application sessions (practicums). Unless otherwise announced, each unit assignment is due on the Friday following its application session. Complete the required preparation and pre-session preparation form before the corresponding Tuesday theory session.

Date	Unit	Session	Topic, activity, or deadline
Thu, Sep 3	—	Introduction	Course welcome, orientation, and semester overview
Tue, Sep 8	1	Theory	Course introduction & framework
Thu, Sep 10	1	Application	Practicum: applying the course framework and setting up the technical workflow
Fri, Sep 11	1	**Deadline**	Unit 1 assignment due
Tue, Sep 15	2	Theory	Data organization & corpus structure
Thu, Sep 17	2	Application	Practicum: constructing traceable corpus tables, units of analysis, and exploratory/confirmatory splits
Fri, Sep 18	2	**Deadline**	Unit 2 assignment due
Tue, Sep 22	3	Theory	Cleaning & sparse translations
Thu, Sep 24	3	Application	Practicum: cleaning text, running diagnostics, and building sparse translations
Fri, Sep 25	3	**Deadline**	Unit 3 assignment due
Tue, Sep 29	4	Theory	What an $n \times m$ matrix enables
Thu, Oct 1	4	Application	Practicum: weighting, normalization, similarity, nearest neighbors, reindexing, and KWIC
Fri, Oct 2	4	**Deadline**	Unit 4 assignment due
Tue, Oct 6	5	Theory	Researcher-defined measurement with sparse representations
Thu, Oct 8	5	Application	Practicum: constructing, auditing, and using transparent text-based measures
Fri, Oct 9	5	**Deadline**	Unit 5 assignment due
Tue, Oct 13	6	Theory	Lower-dimensional representations from sparse matrices: SVD/LSA, topic models, & UMAP
Thu, Oct 15	6	Application	Practicum: generating, inspecting, and comparing lower-dimensional representations
Fri, Oct 16	6	**Deadline**	Unit 6 assignment due
Tue, Oct 20	7	Theory	Predictive supervised learning with sparse representations
Thu, Oct 22	7	Application	Practicum: labeling a subset, fitting, evaluating, and validating a predictive text measure
Fri, Oct 23	7	**Deadline**	Unit 7 assignment due
Tue, Oct 27	8	Theory	Downstream analysis with imperfect surrogate or predicted labels
Thu, Oct 29	8	Application	Practicum: audit samples, naive versus adjusted estimates, bias, and uncertainty
Fri, Oct 30	8	**Deadline**	Unit 8 assignment due
Tue, Nov 3	9	Theory	Learned dense representations: distributional semantics, word2vec, & pretrained encoders
Thu, Nov 5	9	Application	Practicum: training or applying dense representations and revisiting similarity/search tasks

Continued on next page

Course calendar (continued)

Date	Unit	Session	Topic, activity, or deadline
Fri, Nov 6	9	**Deadline**	Unit 9 assignment due
Tue, Nov 10	10	Theory	Linguistic feature engineering: NER, dependency parsing, coreference, SRL, & WSD
Thu, Nov 12	10	Application	Practicum: constructing and locally validating structured linguistic features
Fri, Nov 13	10	**Deadline**	Unit 10 assignment due
Tue, Nov 17	11	Theory	Computer-assisted qualitative text analysis and Geometric Coder (GeCo)
Thu, Nov 19	11	Application	Practicum: strategic reading, retrieval, geometric navigation, coding, memoing, and model assistance in GeCo
Fri, Nov 20	11	**Deadline**	Unit 11 assignment due
Tue, Nov 24	12	Theory	Course synthesis & LLMs; no Unit 12 application session or separate assignment
Thu, Nov 26	—	**No class**	Thanksgiving recess
Tue, Dec 1	—	Review	In-class review and presentations
Thu, Dec 3	—	Review	In-class review and presentations; last class meeting
Thu, Dec 10	—	**Deadline**	Final project due

6 Detailed unit breakdown

All required readings and videos will be on the course site, and supplementary materials can be accessed via HOLLIS. Please let me know if you are unable to find anything.

Unit 1: Course introduction & framework

TL;DR: *Understand CATA as purposeful, computer-assisted translation and learn the AVAIL framework for evaluating translations.*

This unit establishes the course's central framework. Computer-assisted text analysis uses computation to increase both the scale and systematicity of text analysis while retaining human responsibility for interpretation and judgment. We introduce text analysis as purposeful translation: there is generally no uniquely correct representation of a substantive text, only translations that are more or less useful for particular purposes. Students learn AVAIL—Affordances, Validation, Additions, Impositions, and Losses—as the recurring framework for evaluating each translation, including how validation can occur directly or downstream across composed translation chains. We also explain why reading remains necessary and introduce the course's technical environment and Text Analysis Lab (TeAL) as methodological infrastructure for traceable, reproducible workflows.

Unit objectives

- Define computer-assisted text analysis and explain how computation can increase scale and systematicity without making analysis automatic.
- Explain why substantive text admits multiple purpose-dependent translations rather than one uniquely correct computational representation.

- Apply AVAIL—Affordances, Validation, Additions, Impositions, and Losses—to a translation or composed chain of translations.
- Distinguish direct from downstream validation and explain why every substantive claim requires a defensible validity argument somewhere in the relevant analytic chain.
- Describe the continuing role of reading, interpretation, and judgment in computer-assisted text analysis.
- Set up the technical environment for the semester and describe the basic TeAL model of artifacts, translators, and provenance.

Required preparation

VIDEO:	What Is Computer-Assisted Text Analysis?
READING:	Bellos, D. (2011). *Is That a Fish in Your Ear? Translation and the Meaning of Everything*. Particular Books, pp. 4–6, 332–336
VIDEO:	Text Analysis as Translation
VIDEO:	AVAIL: Five Questions for Every Translation
VIDEO:	Why Reading Still Matters
VIDEO:	Introduction to Text Analysis Lab (TeAL)

Supplementary materials

READING:	Markoff, J., Shapiro, G., & Weitman, S. R. (1975). Toward the Integration of Content Analysis and General Methodology. *Sociological Methodology*, *6*, 1–58. https://doi.org/10.2307/270893
READING:	Franzosi, R. (1989). From Words to Numbers: A Generalized and Linguistics-Based Coding Procedure for Collecting Textual Data. *Sociological Methodology*, *19*, 263–298. https://doi.org/10.2307/270955
READING:	Grimmer, J., Roberts, M. E., & Stewart, B. M. (2022). *Text as data: A new framework for machine learning and the social sciences* [OCLC: on1295105650]. Princeton University Press, Chapter 2

Unit assignment
Write a brief memo (1) demonstrating that your computing environment is set up and (2) identifying what kinds of texts you plan/hope to work with and what questions or hopes you have about computer-assisted text analysis at the outset of the semester. Note that this information may be shared with the rest of the class unless you explicitly request otherwise.

Unit 2: Data organization & corpus structure

TL;DR: *Build traceable text corpora with stable identities, appropriate units of analysis, recoverable context, and defensible exploratory/confirmatory structure.*

This unit turns collections of texts into traceable corpora. Students organize texts as identifiable observations in a corpus table built around stable primary keys, text fields, and metadata; define units of analysis that may differ from the units in which texts were originally collected; and preserve hierarchy through compound keys so that lower-level units remain linked to their source context. We then introduce mapping, recomposition, and reindexing as three broad types of translations based on the relationship between source and target artifacts' primary keys. Finally, we consider exploratory/confirmatory splitting as a conditional research-design

choice: it is useful only when the inferential goal, target process, dependence structure, and amount of data make a meaningful held-out confirmation possible.

Unit objectives

- Organize text data into corpus tables with stable primary keys, text fields, and useful metadata.
- Define one or more units of analysis appropriate to a research goal and explain why the source-document unit need not be the analytic unit.
- Use compound primary keys and provenance to preserve hierarchical relationships, recover source context, and reconnect lower-level observations to higher-level metadata.
- Distinguish mapping, recomposition, and reindexing as broad categories of changes to text artifacts.
- Explain when an exploratory/confirmatory split is useful and why splitting should occur at a level that avoids leakage across dependent or nested observations.
- Initialize and manage traceable source and recomposed artifacts within TeAL.

Required preparation

VIDEO: From Texts to a Corpus Table
VIDEO: Units of Analysis
VIDEO: Context and Traceability
VIDEO: Three classes of Translations: Mapping, Recomposition, and Reindexing
VIDEO: Exploratory and Confirmatory Analysis

Supplementary materials

READING: Mosteller, F., & Wallace, D. L. (1964). *Inference and Disputed Authorship: The Federalist*. Addison-Wesley. Retrieved January 31, 2025, from http://archive.org/details/inferencedispute00most Chapters 1–2

READING: Roberts, C. W. (1989). Other Than Counting Words: A Linguistic Approach to Content Analysis. *Social Forces*, *68*(1), 147–177. https://doi.org/10.2307/2579224

READING: Grimmer, J., Roberts, M. E., & Stewart, B. M. (2022). *Text as data: A new framework for machine learning and the social sciences* [OCLC: on1295105650]. Princeton University Press, Chapters 3–4

Unit assignment
Construct and submit an initialized corpus table with metadata and stable unique IDs. If confirmatory inference is part of your intended project and your corpus supports a meaningful split, create and document an exploratory/confirmatory split at the top-level document or source level. Then create at least one additional unit of analysis through recomposition and demonstrate how to recover its source context.

Unit 3: Cleaning & sparse translations

TL;DR: *Treat preprocessing as consequential translation and construct the first sparse $n \times m$ feature representation.*

This unit treats cleaning and preprocessing as consequential translations rather than neutral setup. Students use lightweight corpus diagnostics to learn about the texts and identify observations or patterns that warrant investigation, then consider how tokenization, stop-word filtering, case handling, stemming, lemmatization, and n-grams change what information is retained or collapsed. The unit culminates in the document-term matrix (DTM), where the general idea of an $n \times m$ feature representation is introduced explicitly: n units of analysis are represented using m shared features. The goal is not to find one universally clean corpus or one perfect DTM, but to make preprocessing choices deliberately and understand their consequences.

Unit objectives

- Explain cleaning and preprocessing as purpose-dependent translations rather than invisible or universally correct cleanup.
- Use basic visual and descriptive diagnostics, including log text length, empty-text checks, duplicates, and row-preservation checks, to investigate a corpus.
- Explain tokenization as segmentation into computationally workable units without equating tokens with features.
- Explain how stop-word filtering, lowercasing, stemming, lemmatization, and n-grams remove, collapse, or create distinctions in a textual representation.
- Define a feature and construct document-term matrices in which units of analysis are represented by term features.
- Inspect basic properties of sparse representations, including dimensions, sparsity, vocabulary coverage, and empty rows, while distinguishing feature processing from row subsetting.

Required preparation

VIDEO: Cleaning and Preprocessing as Translations

VIDEO: Basic Corpus Diagnostics

VIDEO: Tokenization and Stop Words

VIDEO: Case, Stemming, Lemmatization, and N-grams

VIDEO: Building a Document-Term Matrix

Supplementary materials

READING: Mosteller, F., & Wallace, D. L. (1964). *Inference and Disputed Authorship: The Federalist.* Addison-Wesley. Retrieved January 31, 2025, from http://archive.org/details/inferencedispute00most Chapters 1–2

READING: Grimmer, J., Roberts, M. E., & Stewart, B. M. (2022). *Text as data: A new framework for machine learning and the social sciences* [OCLC: on1295105650]. Princeton University Press, Chapter 5

Unit assignment

Create at least one cleaned text-field mapping and multiple DTM-style sparse translations from the same source artifact. Submit a short memo describing your basic diagnostics, including log text length, empty or duplicate texts, and row preservation; explain whether any explicit subsetting was needed; and describe how your preprocessing choices shaped the resulting sparse translations.

Unit 4: What an $n \times m$ matrix enables

TL;DR: *Use shared feature spaces for weighting, comparison, nearest-neighbor search, and reindexing.*

This unit develops the main consequences of representing n units of analysis with m shared features. Each row becomes a vector or point in a common feature space, making it possible to reweight and normalize representations, define similarity and distance, retrieve nearest neighbors, and reinterpret the matrix from the perspective of its features. We introduce weighting schemes such as binary representations and TF–IDF, L1/L2 normalization, cosine similarity and distance metrics, and matrix transposition as the simplest example of reindexing. When terms become the units being inspected, corpus-level summaries and keyword-in-context (KWIC) provide a route back from matrix patterns to textual evidence. Throughout, similarity is treated as a property of a particular representation and comparison rule rather than of texts "in themselves."

Unit objectives

- Explain how an $n \times m$ matrix represents observations as vectors in a shared feature space.
- Create and interpret weighted and normalized versions of sparse representations.
- Use similarity and distance metrics to compare observations and retrieve nearest neighbors.
- Explain why similarity is conditional on both the representation and the comparison rule used.
- Explain matrix transposition as a form of reindexing in which original features can become the new units of analysis.
- Inspect term-level distributions and use KWIC/source context to interpret selected patterns and diagnose upstream translation choices.

Required preparation

VIDEO:	Observations in a Feature Space
VIDEO:	Weighting and Normalization as Further Translations
VIDEO:	Similarity, Distance, and Nearest Neighbors
VIDEO:	Reindexing the Matrix: From Documents to Terms

Supplementary materials

READING:	Grimmer, J., Roberts, M. E., & Stewart, B. M. (2022). *Text as data: A new framework for machine learning and the social sciences* [OCLC: on1295105650]. Princeton University Press, Chapter 7
READING:	Jurafsky, D., & Martin, J. H. (2026). *Speech and language processing: An introduction to natural language processing, computational linguistics, and speech recognition with language models* (3rd). https://web.stanford.edu/~jurafsky/slp3/ Section 5.3–5.4

Unit assignment

Using one or more DTMs from Unit 3, create at least one weighted or normalized translation, produce basic unit- and term-level summaries, and run several comparisons or nearest-neighbor searches. Reindex the matrix to inspect terms and use KWIC to investigate at least one pattern in textual context. Write a memo interpreting what these operations reveal and whether they suggest revisions to earlier translator choices.

Unit 5: Researcher-defined measurement with sparse representations

TL;DR: *Construct transparent theory-driven measures from textual features and validate what their measurements capture.*

This unit turns from rich representations to deliberate measurement. Students distinguish a substantive construct from the measure that operationalizes it and the resulting measurements, then study theory-driven measures as a transparent ideal type in which the researcher specifies the mapping from textual features to a construct on substantive grounds rather than learning that mapping from labeled examples. Dictionaries, weighted combinations, and formula-based scores illustrate this approach. Such measures are explicit and reproducible but can be unresponsive to corpus-specific language, so they still require validation. We distinguish validation from iterative tuning: when observed human judgments are repeatedly used to revise the mapping, the procedure is moving toward the supervised-learning logic introduced in the next unit.

Unit objectives

- Distinguish a construct, the measure used to operationalize it, and the resulting measurements.
- Explain why measurement can involve deliberate, substantively purposeful reduction of a richer representation.
- Construct and document transparent theory-driven measures such as dictionaries, weighted combinations, and formula-based scores.
- Explain the strengths of theory-driven measures, including transparency, reproducibility, and direct inspectability, as well as their limited responsiveness to local language patterns.
- Validate theory-driven measurements against higher-quality interpretation or measurement without treating validation as automatic permission for iterative tuning on the same cases.
- Explain why using observed judgments to determine the mapping moves the problem toward supervised learning.

Required preparation

Video:	From Representation to Measurement
Video:	Theory-Driven Measures: Dictionaries and Formulas
Video:	Validating Theory-Driven Measures and Their Limits

Supplementary materials

Reading:	Ho, A. D. (2024). Measurement Must Be Qualitative, then Quantitative, then Qualitative Again. *Educational Measurement: Issues and Practice*, *43*(4), 137–145. https://doi.org/10.1111/emip.12662
Reading:	Jia, J., Miratrix, L., Yu, B., Gawalt, B., Ghaoui, L. E., Barnesmoore, L., & Clavier, S. (2014). Concise comparative summaries (CCS) of large text corpora with a human experiment. *The Annals of Applied Statistics*, *8*(1), 499–529. https://doi.org/10.1214/13-AOAS698
Reading:	Mozer, R., Miratrix, L., Relyea, J. E., & Kim, J. S. (2024). Combining Human and Automated Scoring Methods in Experimental Assessments of Writing: A Case Study Tutorial. *Journal of Educational and Behavioral Statistics*, *49*(5), 780–816. https://doi.org/10.3102/10769986231207886

READING: Jurafsky, D., & Martin, J. H. (2026). *Speech and language processing: An introduction to natural language processing, computational linguistics, and speech recognition with language models* (3rd). https://web.stanford.edu/~jurafsky/slp3/ Chapter 22

Unit assignment

Using a sparse translation of your corpus, construct at least two pre-specified, substantively defensible researcher-defined measures of meaningful constructs. Document the theoretical mapping from features to scores and compare how the alternative specifications change the resulting measurements. Then audit selected high-, low-, borderline-, or otherwise informative cases against source context and explain what the validation reveals about the measures' strengths and limits.

Unit 6: Unsupervised learning for dimensionality reduction: SVD/LSA, topic models, UMAP, & clustering

TL;DR: *Create lower-dimensional representations that can consolidate signal, improve generalization, and make high-dimensional structure easier to inspect.*

This unit transitions from high-dimensional sparse matrices to lower-dimensional, denser representations using unsupervised learning techniques. Dimensionality reduction can consolidate redundant co-occurrence patterns and discard weaker variation, sometimes yielding cleaner signals that generalize beyond literal feature overlap; it also makes high-dimensional structure easier to inspect and visualize. We contrast three families of sparse-to-dense translations. SVD/LSA algebraically finds ranked low-dimensional structure already present in the input matrix; LDA/topic models fit a probabilistic generative structure in which documents are mixtures of topics and topics are distributions over words; and UMAP constructs low-dimensional projections designed to preserve local neighborhoods. We also introduce the more general concept of clustering. Each method is evaluated relative to its immediate input using AVAIL rather than treating all dense outputs as interchangeable embeddings.

Unit objectives

- Explain why dimensionality reduction can support generalization, consolidation of redundant signal, and visualization while necessarily discarding some variation.
- Describe SVD/LSA as a ranked low-rank representation of common structure in the input matrix that does not import external corpus information.
- Describe LDA/topic models as probabilistic generative representations with stronger structural impositions than SVD/LSA but no necessary addition of external information.
- Explain UMAP as a neighborhood-oriented projection that prioritizes local relationships and can distort global geometry.
- Apply AVAIL to compare what alternative lower-dimensional translations afford, validate, add, impose, and lose relative to the sparse representations from which they are derived.
- Return from low-dimensional dimensions, topics, neighborhoods, and outliers to source texts when evaluating substantive patterns.

Required preparation

VIDEO: Why Reduce Dimensionality?

VIDEO: SVD and Latent Semantic Analysis

VIDEO: Latent Dirichlet Allocation and Topic Models

VIDEO: UMAP and Neighborhood-Preserving Projection

Supplementary materials

READING: [DRAFT] Lim, Z. (2026). *Interacting with Topic Models.*

READING: Deerwester, S., Dumais, S. T., Furnas, G. W., Landauer, T. K., & Harshman, R. (1990). Indexing by latent semantic analysis. *Journal of the American Society for Information Science*, *41*(6), 391–407. https://doi.org/10.1002/(SICI)1097-4571(199009)41:6<391::AID-ASI1>3.0.CO;2-9

READING: Blei, D. M., Ng, A. Y., & Jordan, M. I. (2003). Latent Dirichlet Allocation. *Journal of Machine Learning Research*, *3*, 993–1022

READING: Roberts, M. E., Stewart, B. M., Tingley, D., & Airoldi, E. M. (2013). The Structural Topic Model and Applied Social Science

Unit assignment

Create at least two lower-dimensional representations derived from a sparse representation of your corpus. Compare the structures they reveal, inspect selected neighborhoods, dimensions, topics, or outliers, and return to the source texts to evaluate whether the resulting patterns are substantively meaningful. Use AVAIL to explain at least one important difference between the representations.

Unit 7: Predictive supervised learning with sparse representations

TL;DR: *Learn mappings from labeled examples, evaluate whether they generalize, and use them to scale judgments to unlabeled texts.*

This unit introduces predictive supervised learning as an alternative to researcher-specified measurement rules. Instead of fixing the feature-to-measurement mapping in advance, researchers provide labeled examples and a learner estimates a mapping that can extend those judgments to unlabeled cases. Supervised learning is a broad family: researchers choose which cases to label—including random sampling or active-learning strategies—and how the model learns from them. We emphasize generalization rather than algorithmic internals: training performance can be misleading because a model may overfit the cases from which it learned, so predictive measurements must be evaluated on separately labeled cases that did not determine the fitted model.

Unit objectives

- Distinguish predictive goals from descriptive and inferential goals at a practical level.
- Explain supervised learning as learning a mapping from textual features to human-provided labels or measurements and extending that mapping to unlabeled cases.
- Explain that supervised learning includes choices about which cases to label, including random sampling and active learning, as well as choices among learning algorithms.
- Explain overfitting and why training performance does not establish generalization to new cases.
- Distinguish training/development examples from separately labeled evaluation cases.

- Interpret confusion matrices, accuracy, precision, recall, false positives, and false negatives in relation to substantively important errors.

Required preparation

VIDEO: What Is Predictive Supervised Learning?

VIDEO: Training a Classifier

VIDEO: Validating Predictive Text Measures

Supplementary materials

READING: Hillard, D., Purpura, S., & Wilkerson, J. (2008). Computer-Assisted Topic Classification for Mixed-Methods Social Science Research. *Journal of Information Technology & Politics*, *4*(4), 31–46. https://doi.org/10.1080/19331680801975367

READING: D'Orazio, V., Landis, S. T., Palmer, G., & Schrodt, P. (2014). Separating the Wheat from the Chaff: Applications of Automated Document Classification Using Support Vector Machines. *Political Analysis*, *22*(2), 224–242. https://doi.org/10.1093/pan/mpt030

READING: Wiedemann, G. (2019). Proportional Classification Revisited: Automatic Content Analysis of Political Manifestos Using Active Learning. *Social Science Computer Review*, *37*(2), 135–159. https://doi.org/10.1177/0894439318758389

READING: Jurafsky, D., & Martin, J. H. (2026). *Speech and language processing: An introduction to natural language processing, computational linguistics, and speech recognition with language models* (3rd). https://web.stanford.edu/~jurafsky/slp3/ Chapter 4

Unit assignment

Using labels from a subset of your exploratory set, fit and evaluate a predictive classifier using a sparse translation of your corpus or the shared course corpus. Submit a memo describing how cases were selected for labeling, how separately labeled cases were reserved for evaluation, and what the validation reveals about generalization and important error types. You do not need to describe technical training or tuning procedures not covered in the course; identify the TeAL learner and settings used and focus on interpreting its evaluation results.

Unit 8: Downstream analysis with imperfect measurements

TL;DR: *Use imperfect text-derived measurements responsibly in downstream descriptive and inferential analysis.*

This unit asks what happens when imperfect measurements become inputs to substantive downstream analyses. The problem is broader than predicted labels: theory-driven rules, supervised models, LLM-based measures, and other surrogates can all be systematically wrong. Even highly accurate measures can badly bias means, prevalence estimates, group differences, treatment effects, or regression relationships when errors are asymmetric or differ across groups. We therefore introduce designed audit samples in which a subset of cases receives higher-quality measurements. When the selection design is known, these audit measurements can be used to estimate systematic error, adjust downstream quantities, and represent the additional uncertainty introduced by imperfect measurement.

Unit objectives

- Explain why high measurement or predictive accuracy does not by itself guarantee trustworthy downstream descriptive or inferential quantities.
- Explain how systematic and differential measurement error can create, attenuate, exaggerate, or reverse downstream patterns, including group differences.
- Describe the role of higher-quality audit measurements obtained on a sample selected through a known probability design.
- Explain, at a high level, how audit information can be used to estimate measurement error, debias downstream estimands, and account for uncertainty.
- Recognize that the same downstream-error logic applies to theory-driven, machine-learned, LLM-derived, and other imperfect measurements.

Required preparation

VIDEO: Using Imperfect Measurements in Downstream Analysis

VIDEO: Auditing, Debiasing, and Uncertainty

Supplementary materials

READING: Egami, N., Hinck, M., Stewart, B., & Wei, H. (2023). Using Imperfect Surrogates for Downstream Inference: Design-based Supervised Learning for Social Science Applications of Large Language Models. *Advances in Neural Information Processing Systems*, *36*, 68589–68601. Retrieved May 7, 2025, from https://proceedings.neurips.cc/paper_files/paper/2023/hash/d862f7f5445255090de13b825b880d59-Abstract-Conference.html

READING: Angelopoulos, A. N., Bates, S., Fannjiang, C., Jordan, M. I., & Zrnic, T. (2023). Prediction-powered inference. *Science*, *382*(6671), 669–674. https://doi.org/10.1126/science.adi6000

Unit assignment

Using provided imperfect measurements together with higher-quality measurements from a designed audit sample, compare a naive downstream estimate with an uncertainty-aware or debiased estimate of a simple target such as a mean, group difference, or regression coefficient. Submit a short memo explaining the estimand, why systematic measurement error may bias or overstate confidence in the naive analysis, what information the correction uses, and how the adjusted inference changes the substantive conclusion.

Unit 9: Learned dense representations: distributional semantics, Word2Vec, & pretrained encoders

TL;DR: *Learn how prediction produces dense language representations, reuse them as feature spaces, and evaluate the validity of measurement claims made from their geometry.*

This unit moves from dense representations derived from a local sparse matrix to representations learned through prediction on language itself. The distributional hypothesis links linguistic meaning to patterns of contextual use; Word2Vec turns that intuition into predictive learning that produces static word embeddings whose geometry can encode both similarity and recurring relationships. We then move to high-capacity contextual models such as BERT, where masked-language prediction and large-scale pretraining produce context-sensitive representations that can transfer to many downstream tasks. Sentence-level encoders turn texts into dense $n \times m$

feature matrices that can be used with familiar tools such as similarity, nearest-neighbor search, visualization, and supervised learning. The unit closes by treating pretrained geometry as a measurement problem: cosine similarity in an opaque, externally learned space is not automatically a valid measure of a substantive construct.

Unit objectives

- Explain the distributional hypothesis and connect it to earlier co-occurrence-based representations such as LSA.
- Explain how Word2Vec learns static word embeddings through predictive tasks and how learned geometry can encode both similarities and recurring relationships among words.
- Distinguish locally trained and pretrained static embeddings from contextual representations whose outputs depend on surrounding text.
- Explain masked-language modeling, pretraining, and fine-tuning at a conceptual level and why large contextual models require extensive external data and computation.
- Treat sentence/document embeddings as dense $n \times m$ feature spaces that can support similarity, retrieval, visualization, and supervised learning.
- Explain why pretrained embedding similarity is model-dependent and requires a validity argument before it is interpreted as substantive measurement.
- Apply AVAIL to distinguish external information added through pretraining from the structural impositions of the model and the losses/opacity of the resulting latent representation.

Required preparation

VIDEO: The Distributional Hypothesis

VIDEO: Word2Vec: Learning Representations from Prediction

VIDEO: From Static Word Vectors to Pretrained Contextual Representations

VIDEO: Dense Representations as Feature Spaces

VIDEO: Measurement with Pretrained Representations

Supplementary materials

READING: Mikolov, T., Chen, K., Corrado, G., & Dean, J. (2013, September). Efficient Estimation of Word Representations in Vector Space [arXiv:1301.3781 [cs]]. https://doi.org/10.48550/arXiv.1301.3781

READING: Arora, S., Liang, Y., & Ma, T. (2017). A Simple but Tough-to-Beat Baseline for Sentence Embeddings. Retrieved May 8, 2026, from https://openreview.net/forum?id=SyK00v5xx

READING: Garg, N., Schiebinger, L., Jurafsky, D., & Zou, J. (2018). Word embeddings quantify 100 years of gender and ethnic stereotypes. *Proceedings of the National Academy of Sciences*, *115*(16). https://doi.org/10.1073/pnas.1720347115

READING: Kozlowski, A. C., Taddy, M., & Evans, J. A. (2019). The Geometry of Culture: Analyzing the Meanings of Class through Word Embeddings. *American Sociological Review*, *84*(5), 905–949. https://doi.org/10.1177/0003122419877135

READING: Rogers, A., Kovaleva, O., & Rumshisky, A. (2020, November). A Primer in BERTology: What we know about how BERT works [arXiv:2002.12327 [cs]]. https://doi.org/10.48550/arXiv.2002.12327

Unit assignment

Revisit an earlier analytic task, such as nearest-neighbor retrieval, similarity search, visual-

ization, or supervised prediction, using a learned dense representation, including a pretrained representation. Compare the results with an earlier sparse version of the same task. Submit a memo explaining what each representation affords, what external information it adds, what structure it imposes or obscures, and how the resulting patterns or measurements can be validated for the intended use.

Unit 10: Linguistic feature engineering: NER, dependency parsing, coreference, SRL, & WSD

TL;DR: *Construct traceable, task-relevant features from comparatively explicit linguistic structure and validate them against source context.*

This unit moves from generic sparse features and opaque latent dense representations to deliberately engineered linguistic structure. Named entity recognition, dependency parsing, coreference resolution, semantic role labeling, and word-sense disambiguation expose entities, grammatical relations, reference, event roles, and context-sensitive word meanings that bags of words and dense embeddings do not directly expose. These model outputs are not yet the substantive representation: researchers decide which linguistic information is useful and how to translate it into traceable engineered features. The unit also highlights an important AVAIL distinction. Linguistic translators often add general linguistic information learned outside the local corpus, but many of their explicit outputs can be validated directly against source context and knowledgeable human judgment.

Unit objectives

- Explain linguistic feature engineering as the deliberate construction of task-relevant representations from explicit linguistic model outputs.
- Identify the kinds of information exposed by named entities, dependency relations, coreference, semantic roles, and word senses.
- Distinguish linguistic translator outputs from the engineered feature representation a researcher builds from those outputs.
- Select linguistic translators according to the substantive information a research task requires rather than using them merely because they are available.
- Explain how linguistic translators can add general linguistic information or competence from outside the local corpus.
- Validate explicit linguistic outputs and engineered features against local source context, identify failure modes, and explain how errors can propagate through composed translations.
- Preserve provenance so engineered features can be traced through intermediate linguistic outputs back to the original text.

Required preparation

Video:	From Linguistic Structure to Engineered Features
Video:	Entities, Dependencies, and Coreference
Video:	Semantic Roles, Word Senses, and Meaning in Context
Video:	Combining and Validating Linguistic Features

Supplementary materials

READING: Ash, E., Gauthier, G., & Widmer, P. (2024). Relatio: Text Semantics Capture Political and Economic Narratives. *Political Analysis*, *32*(1), 115–132. https://doi.org/10.1017/pan.2023.8

READING: [DRAFT] Lim, Z. (2026). *Bag of Ideas*.

Unit assignment

Engineer a small set of structured linguistic features for a subset of your corpus using one or more linguistic translators. Explain why the selected linguistic information is useful for your substantive task, inspect the translator outputs and resulting features against their source context, identify important failure modes or propagated errors, and document how the engineered features remain traceable to the original text.

Unit 11: Computer-assisted qualitative inquiry and geometric coding

TL;DR: *Use the course's existing representations and analytic tools to make serious qualitative inquiry more scalable and systematic without replacing interpretation.*

This unit synthesizes rather than introduces a new computational toolbox. We begin by taking qualitative research seriously as a rich methodological tradition of close reading, comparison, coding, memoing, interpretation, conceptual development, and attention to anomalies and surprises. That depth can create genuine trade-offs with scale and systematicity. Geometric coding asks how the methods developed throughout the course can help address those trade-offs: instead of treating a corpus as an unordered collection of texts, researchers can construct one or more intentional geometries in which textual units occupy positions relative to one another. Neighbors, retrieval, projections, supervised learning, active learning, validation, and context recovery then become tools for allocating interpretive attention. Geometric Coder (GeCo) brings these existing components together in an exploratory environment while preserving reading, coding, memoing, and source context as central human activities.

Unit objectives

- Explain why qualitative inquiry is not reducible to categorical labeling and identify the methodological value of close reading, comparison, coding, memoing, interpretation, and conceptual development.
- Explain the trade-offs that rich human interpretation can face with scale and systematicity without treating those trade-offs as evidence that qualitative inquiry is methodologically weak.
- Explain how a computational representation can organize textual units into a geometry that supports strategic comparison, retrieval, navigation, and case selection.
- Use AVAIL to design and compare multiple candidate geometries rather than treating any single embedding space as the corpus's natural or uniquely correct geometry.
- Explain how familiar tools such as nearest neighbors, vector search, UMAP, supervised learning, and active learning can assist qualitative exploration by allocating human attention rather than supplying self-interpreting findings.
- Use GeCo to integrate geometric navigation, contextual reading, coding, memoing, model assistance, validation, and traceability in an iterative exploratory workflow.

Required preparation

VIDEO: Taking Qualitative Research Seriously

VIDEO: From a Corpus to a Geometry

VIDEO: Designing Geometries for Qualitative Inquiry

VIDEO: GeCo: Bringing the Pieces Together

Supplementary materials

READING: Nelson, L. K. (2020). Computational Grounded Theory: A Methodological Framework. *Sociological Methods & Research*, *49*(1), 3–42. https://doi.org/10.1177/0049124117729703

READING: Carlsen, H. B., & Ralund, S. (2022). Computational grounded theory revisited: From computer-led to computer-assisted text analysis. *Big Data & Society*, *9*(1), 1–16. https://doi.org/10.1177/20539517221080146

READING: [DRAFT] Lim, Z. (2026). *The Latent Interview Protocol: Recovering Enacted Question Structure for Large-Scale Interview Analysis.*

Unit assignment

Use GeCo to conduct a small computer-assisted qualitative exploration of your corpus. Construct or select one or more geometries intentionally, then use some combination of retrieval, geometric navigation, contextual reading, coding, memoing, active learning, or model assistance to identify and investigate substantively interesting cases or patterns. Submit a memo explaining how the geometry and computational tools shaped where you looked, what you learned through contextual reading and interpretation, and how the workflow improved scale or systematicity without replacing qualitative judgment.

Unit 12: Course synthesis & large language models

TL;DR: *Synthesize CATA as purposeful chained translation and evaluate LLMs as powerful but methodologically ordinary translators and measurement devices.*

The final unit retells the semester as one coherent story of purposeful, chained translations evaluated through AVAIL, then places large language models within the same framework rather than treating them as a methodological exception. LLMs are very large learned translators: text is tokenized, transformed through many opaque learned representations, and used to predict further text. Their extensive pretraining gives them remarkable flexibility but also imports large amounts of external information and opacity. We focus especially on LLMs used inside the analytic pipeline as measures: a prompt, model, and procedure together produce measurements that can contain systematic error and therefore require local validation, auditing, and, when appropriate, downstream debiasing. The unit closes with tool choice: LLMs can be valuable research assistants and measures, but simpler, cheaper, more transparent translators are preferable when they adequately serve the research purpose, and exploratory work should preserve the researcher's direct engagement with source material.

Unit objectives

- Synthesize the semester as a chain of purposeful translations and use AVAIL to reason about their cumulative affordances, validation, additions, impositions, and losses.
- Explain an LLM at a high level as a large learned chain of tokenization, numerical representation, hidden transformations, and next-token prediction rather than as a methodological exception to CATA.

- Explain how pretraining gives LLMs flexible general-purpose capabilities while importing substantial external information and opacity into an analytic pipeline.
- Treat an LLM used to produce judgments, labels, comparisons, or scores as a measure whose prompt, model, and procedure must be specified and empirically evaluated.
- Explain why LLM measurements can contain systematic error and how designed audits and downstream correction can be used when those measurements enter substantive analyses.
- Distinguish general-purpose LLM assistance outside the substantive analytic pipeline from delegating measurement, extraction, or synthesis inside the pipeline.
- Evaluate whether an LLM is appropriate for a task or whether a simpler, more transparent, cheaper, or more directly validatable translator would be preferable.
- Explain why delegating exploratory interpretation to a pretrained black box can constrain the researcher's encounter with the data even when the model can generate useful hypotheses.

Required preparation

VIDEO: Computer-Assisted Text Analysis in One Story

VIDEO: Large Language Models Are Translators

VIDEO: Large Language Models as Measures

VIDEO: When Should You Use an LLM?

Supplementary materials

READING: Halterman, A., & Keith, K. A. (2025). Codebook LLMs: Evaluating LLMs as Measurement Tools for Political Science Concepts. *Political Analysis*, 1–17. https://doi.org/10.1017/pan.2025.10017

Unit assignment

No separate Unit 12 assignment. Use this week to prepare your final project and integrative reflection. Your final submission should explain the major translators and representations you used, apply AVAIL to the consequential steps in the pipeline, describe where validation occurred directly or downstream, and explain how the pieces fit together into a defensible substantive analysis.